



**ANGELES UNIVERSITY FOUNDATION**  
*Angeles City*  
**COLLEGE OF COMPUTER STUDIES**  
***1st Semester, A.Y. 2026-2027***



**Consistency by Design: Controlling Instantiation with the Singleton**

**Pattern in Modern PHP**

Midterm Requirement

**IPT10 – Integrative Programming and Technologies**

**Santos, Jakov Benedict B.**

BSIT-3-A

Submitted to:

**Mr. Romack L. Natividad, MIT**

September 25, 2026

## 1. Introduction and Problem Statement

Some objects in a program should only exist once, no matter how many times the program needs them. A settings file, a logging tool that writes to one log, or a manager that controls access to a shared resource are all good examples. If different parts of a program are each allowed to make their own copy of one of these objects, the program ends up with several separate versions of something that was supposed to be a single shared source of truth. The Singleton pattern, one of the twenty-three patterns catalogued by the *Gang of Four* (GoF), was created to solve exactly this problem. Its original definition is to *"ensure a class only has one instance, and provide a global point of access to it"* (Gamma, Helm, Johnson, & Vlissides, 1994, p. 127). In plain terms, no matter where in the program you ask for it, you always get back the same object.

The problem becomes clearer with a real example. Imagine a small PHP program that reads its settings, such as where exported files should be saved, from a configuration file. Consider this program uses a class called ``AppSettings`` to hold those settings. Without a Singleton, any part of the program is free to create a brand-new ``AppSettings`` object whenever it needs one, and each new copy reads the configuration file again from scratch. If one part of the program later changes a setting while it is running, such as switching the save folder partway through, the other copies never find out because they are separate objects with their own state. To work around this, developers may end up adding improvised shared lists, checks, or other mechanisms across the codebase to keep the settings synchronized. Without a centralized instance, every class that needs the same configuration must either manage its own copy or depend on another mechanism for sharing it. This adds unnecessary coordination and makes it easier for different parts of the program to work with inconsistent configuration state. The Singleton pattern addresses this problem by providing one shared instance of ``AppSettings``, giving different parts of the program a single source for accessing the same configuration state.

Skipping the pattern altogether causes three kinds of trouble. First, the program wastes time and memory building the same object and reading the same file, over and over. Second, the program's state becomes inconsistent since updating one copy does nothing to the rest. Third, bugs caused by this mismatch are hard to trace, because nothing about any single class reveals that other, disconnected copies even exist. The Singleton pattern addresses these problems by placing object creation and access behind a single shared method, which is the structure this article looks at next.

## 2. Literature and Theoretical Grounding

The Singleton pattern's authority as a design solution starts with the original definition written by Gamma, Helm, Johnson, and Vlissides (1994). They grouped it with the other "creational" patterns, patterns that deal with how objects get created, laid out its structure across pp. 127–134 of their book: one hidden field that stores the single object, a hidden constructor so nothing outside the class can create a new copy, and one public method that builds the object the first time it is asked for and simply hands back the same one after that. This original text is still the reference point that every later, PHP-specific explanation builds on.

A simple analogy is a school's central announcement board. Different students may need to check the same information, but creating a separate board for every student would defeat the purpose of having one shared source. In the same way, `AppSettings` provides one shared configuration object that different parts of the program can access.

Freeman et al. (2020) describe the same structure in a more beginner-friendly way in their chapter One of a Kind Objects in *Head First Design Patterns*. They focus on two important issues: creating the object only when it is first needed and avoiding the possibility of multiple instances when concurrent threads request it at the same time. Their explanation is useful for building intuition, but it is not written with any one language in mind. In a conventional PHP request or command-line execution, the Singleton instance exists only for the lifetime of that PHP process.

Zandstra (2016), in the fifth edition of *PHP Objects, Patterns, and Practice*, fills that gap by walking through the pattern directly in PHP. He shows how to mark the constructor private so no other code can create a new object directly, how to block copying with PHP's `__clone()` method, and how Composer's autoloading and namespace rules affect where a Singleton class should live in a project. These practical details directly shaped the PHP 8.1 example built for this article.

Beyond the two textbooks, real research gives the pattern a more critical look. Ampatzoglou et al. (2015) studied close to 65,000 open-source Java classes to see how much each class was affected by changes happening elsewhere in the code, a quality they call stability. They found that a class's stability was measurably different depending on which design pattern role it played, including the Singleton role, and how many times that pattern was used in the class. Their main takeaway, that a pattern's effect on code quality depends on the situation rather than being automatically good, is a useful counterweight to the mostly positive tone of the two textbooks above.

Finally, Shvets (n.d.), writing on Refactoring. Guru, shows where the pattern stands in current developer opinion. Many developers now see Singleton as close to a bad practice, since it behaves like a global variable any part of the program can reach and change, which is exactly what makes automated tests hard to trust. Together, these five sources move from the original definition (Gamma et al.), through pedagogy (Freeman et al.), PHP practice (Zandstra), and evidence (Ampatzoglou et al.), to today's developer opinion (Shvets), the same order this article follows below.

### 3. Structure and Diagrams

Figure 1 shows the canonical structure of the Singleton pattern: the Singleton class keeps a reference to itself in a static field, exposes that reference only through `getInstance()`, and hides its constructor so that a Client can never call `new` directly. Every request for the object, from anywhere in the codebase, is routed through the same accessor and therefore resolves to the same object in memory.

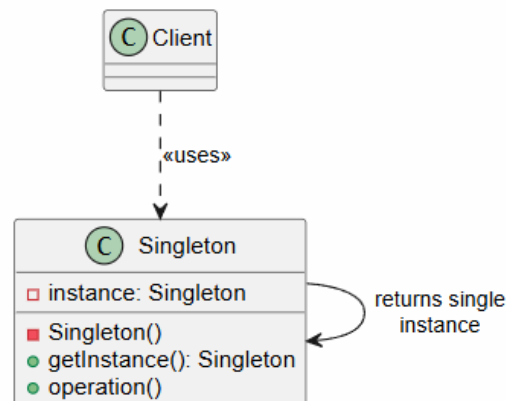


Figure 1. UML class diagram of the Singleton pattern. Source: Author's own work, generated using PlantUML.

Figure 2 applies this structure to the concrete example used throughout this article: the `AppSettings` singleton in the author's own `FileFlow Report Exporter`, a PHP console project built for an earlier IPT10 laboratory activity. When the `ExportCommand` first asks for `AppSettings::getInstance()`, the class checks whether an instance already exists; on the first call, it does not. So one is created and cached. Every subsequent call, whether from the same command or from `TxtExporter` and `JsonExporter`, simply receives the cached object, so the configuration file is parsed exactly once per run instead of once for every separate `AppSettings` object.

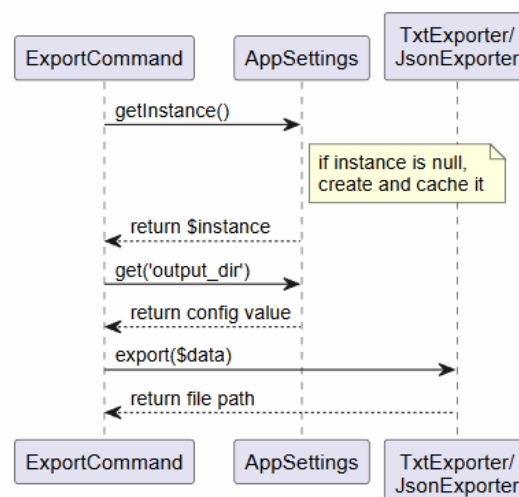


Figure 2. Sequence diagram illustrating the Singleton implementation in `FileFlow Report Exporter`. Source: Author's own work, generated using PlantUML.

## 4. PHP Implementation

The code below is my own PHP 8.1 version, written for the FileFlow Report Exporter, not copied from any textbook. The AppSettings class plays the Singleton role shown in Figure 1. Its constructor is marked private, so no other part of the program can create an AppSettings object directly. The class itself is marked final, blocking a modified version that could sneak in as a second way to build the object, and copying is blocked too, through `__clone()`. The `getInstance()` method does the check shown in Figure 2: it checks whether the object already exists and only builds one the first time it does not. The class stays small on purpose, exposing only `get()` in this implementation, so other parts of the program can read a setting without changing it directly.

```
<?php
declare(strict_types=1);
namespace FileFlow\Config;

final class AppSettings
{
    private static ?AppSettings $instance = null;
    private array $values; // @var array<string, string>

    // Private constructor blocks `new AppSettings()` from outside.
    private function __construct(string $configPath)
    {
        if (!is_file($configPath)) {
            throw new \RuntimeException("Missing config: {$configPath}");
        }
        $this->values = parse_ini_file($configPath) ?: [];
    }
    private function __clone(): void
    {
    } // blocks copy-by-clone
    public function __wakeup(): void // blocks copy-by-unserialize
    {
        throw new \RuntimeException('Cannot unserialize AppSettings.');
```

The `composer.json` file below tells Composer, PHP's class-loading tool, where to find the FileFlow\ classes: inside `src/`.

```
{
  "name": "jakov/fileflow-pattern-lab",
  "autoload": {
    "psr-4": { "FileFlow\\": "src/" }
  },
  "require": { "php": ">=8.1" }
}
```

## 5. Application and Reflection

Inside FileFlow Report Exporter, `AppSettings` is the only place where the project's configuration file is read. The console entry point resolves the settings once through `getInstance()`, and both the TXT and JSON exporters, built with the Factory Method pattern, use the same instance to obtain the output directory before writing a file. Because Composer's PSR-4 autoloader resolves the class when it is needed and PHP keeps the instance within the lifetime of the current CLI process, this provides a smaller-scale example of the same shared-instance idea used in frameworks. In Laravel, for example, the service container allows a developer to register a class or interface as a singleton binding so that subsequent resolutions return the same instance. This allows different parts of the application to access the same shared service through the container instead of each component constructing its own separate copy.

| When to Use                                                                                                                  | When NOT to Use                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Exactly one shared resource must exist for the life of the process (a config store, a connection pool, a single log writer). | The “one instance” rule is really just a convenience for passing an object around; dependency injection expresses that more honestly. |
| Global read access is genuinely needed, and the object holds no request-specific or user-specific state.                     | The class will be unit tested in isolation; a hidden global instance makes tests depend on execution order.                           |
| The cost of re-creating the object (e.g., re-parsing a file) is high enough to justify caching it once per run.              | The application may need more than one instance later (e.g., multi-tenant configs); Singleton makes that change harder.               |

Without this pattern, FileFlow's exporters would each open and parse `config/app.ini` independently. In a small student project, that only wastes a few file reads, but the same approach in a larger application, where dozens of classes may hold slightly different snapshots of configuration, can create inconsistencies that are difficult to reproduce because they depend on which copy was read or modified. Working through this pattern also clarified its most common criticism for me: because `AppSettings` is reachable from anywhere through a static call, it can behave like a global variable, and if I ever need to unit test `ExportCommand` against a fake configuration, the static `getInstance()` makes that awkward unless the class is refactored to accept the settings object as a constructor argument instead. That trade-off between easy global access and harder testing is the main lesson this exercise reinforced. For this small console tool, Singleton provides a straightforward way to maintain a single shared configuration instance, but testing and flexibility concerns become more important as the codebase grows. The empirical caution raised by Ampatzoglou et al. (2015) and the practitioner skepticism noted by Shvets (n.d.) support considering dependency injection instead when the application becomes larger or requires more flexible testing.

## References

- Ampatzoglou, A., Chatzigeorgiou, A., Charalampidou, S., & Avgeriou, P. (2015). The effect of GoF design patterns on stability: A case study. *IEEE Transactions on Software Engineering*, 41(8), 781–802. <https://doi.org/10.1109/TSE.2015.2414917>
- Freeman, E., Robson, E., Sierra, K., & Bates, B. (2020). Head first design patterns: Building extensible and maintainable object-oriented software (2nd ed.). O'Reilly Media. ISBN 978-1-4920-7800-5.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. ISBN 0-201-63361-2.
- Shvets, A. (n.d.). Singleton in PHP. Refactoring.Guru. Retrieved September 19, 2026, from <https://refactoring.guru/design-patterns/singleton/php/example>
- Zandstra, M. (2016). PHP objects, patterns, and practice (5th ed.). Apress. <https://doi.org/10.1007/978-1-4842-1996-6>

### GitHub Repository:

[https://github.com/JakovSantos/IPT10\\_MIDTERM\\_Santos\\_JakovBenedict\\_Singleton](https://github.com/JakovSantos/IPT10_MIDTERM_Santos_JakovBenedict_Singleton)